

DON'T PANIC

Bloated MP3 Player — Deadline Analysis

From 9 concurrent peripherals on ESP32-S3 to provable scheduling guarantees.
Rate-Monotonic · EDF · PIP / PCP · Processor Demand Criterion

Henry Letellier · ESP32-S3 WROOM (dual-core 240 MHz) · FreeRTOS via Arduino
7 slides · No firmware modified · Analysis from `bonus/rta/*.py`
`bonus/rta/lenv` · python-pptx + matplotlib · 2026-09-01

WHAT THIS DECK PROVES

U & Bound	$U=0.947 > 0.757 \rightarrow$ RTA needed
RTA	b,c,d OK · a $R=41 > 40$ fails RM
EDF	$U=0.617 \leq 1$ and PDC passes (600 pts)
Blocking	PCP halves B vs PIP (10→4, 10→6)
Complexity	Busy period $L^*=10$: 600→1 point (600×)

*The Hitchhiker's answer was 42.
Our answer is 40: the period where RM breaks.*

Tip: wave at the HC-SR04 to pause.
Wave twice to bail out — just like deadline handling.

Bloated by design, real-time by necessity

ESP32-S3 dual-core · FreeRTOS · 9 tasks · “17 sensors on one I²C bus”

HARDWARE — THE BOM OF DOOMED PERIPHERALS

Display	GMG12864-06D 128×64 SPI (ST7565R) CS10 RST9 DC8
Char LCDs	2004A (0x27) folder/track + 1602A (0x26) Vagon poetry — I ² C SDA2/SCL1
Audio	2× MAX98357A I ² S BCLK16 WS47 DIN17/18 — Helix MP3 + PSRAM
Sensors	AHT20+BMP280 (env) · MPU-9250 (IMU) · HC-SR04 (TRIG14 ECHO13)
Input / Light	EC11 rotary A42 B15 SW12 · WS2812B 16×16 matrix pin 41
Storage	microSD SDMMC 1-bit CLK39 CMD38 D0 40
Compute	Freenove ESP32-S3 WROOM dual-core 240 MHz · Arduino + FreeRTOS

FREERTOS MAPPING — CORE & STACK REALITY

Task	Prio	Core	Stack	Note
Audio	3	0	16 384	Helix + FATFS deep chain
Input	2	0	16 384	SD open() overflow fix
UI / LED / Matrix	1	1	4–8K	33 ms UI, I ² C traffic
Sensor / Serial / Vagon	1	0/1	4–8K	I ² C 50 ms timeout, BLE GATTS

Lesson: Audio on core 0 prio 3 isolates decode; everything else 1 — dual-core = interference domain partition.

WHY THIS NEEDS ANALYSIS — CONCURRENCY REALITY

The joke: 17 peripherals arguing on one I²C bus.

- FreeRTOS preemptive scheduler ticks every 1 ms — 9 tasks compete, priorities decide who preempts whom.
- Shared buses: I²C (AHT20/BMP280 + 2 char LCDs), SPI (LCD + font ROM), SDMMC (FATFS). All need bounded blocking.
- Stacks matter: printf(%a.1f) pulls _dtoa_r + malloc; Audio/Input blew 8K → 16K canary. Not in the model, but real.
- Audio cannot starve: double-buffered I²S with Helix fixed-point decode; Input (SD open) runs near-Audio depth — hence prio 2.
- Question for this deck: if we idealise 4 tasks, can RM guarantee deadlines? If not, does EDF? At what cost?

Takeaway: model 4 periodic tasks (b,c,d,a) to stress the method, then map back to the 9-task dual-core reality → slide 7.

From 9 threads to 4 canonical tasks

Enough to prove the point, small enough to compute by hand — then verify with bonus/rta

TASK SET — RM ORDER (shortest period = highest priority)

Task	C (WCET)	T = D	Prio (RM)	U = C/T
τ_b	2	8	1	0.250
τ_c	3	10	2	0.300
τ_d	4	18	3	0.222
τ_a	7	40	4	0.175
Total utilisation $U = \sum C/T$				0.947

D = T for all tasks (implicit deadlines). Solver: bonus/rta/rta_solver.py

UTILISATION BOUND — SUFFICIENT, NOT NECESSARY

Liu & Layland (1973) — $U \leq n(2^{1/n} - 1)$

n = 4
Bound = $4 \cdot (2^{1/4} - 1)$

= 0.757

Actual U = 0.947
U > Bound (0.947 > 0.757)

INCONCLUSIVE → RTA

Bound is pessimistic: harmonic periods pass more often. RTA (next slide) is exact — iterate $R = C + \sum \text{ceil}(R/T_h) \cdot C_h$ until fixed point or deadline miss.

WHAT RTA ACTUALLY COMPUTES — exact response-time iteration

$R_i^0 = C_i$ $R_i^{k+1} = C_i + \sum_{h \in \text{hp}(i)} \text{ceil}(R_i^k / T_h) \cdot C_h$

Stop when $R^{k+1} = R^k$ (schedulable if $\leq D$) or $R > D$ (miss). Each iteration adds interference from all higher-priority jobs that fit in the window.

INTUITION: “How long am I delayed by everyone above me?”

- 03 / 07
- $\text{hp}(i)$: higher-priority tasks
 - $\text{ceil}(R/T_h)$: how many jobs of h interfere
 - Converges in $\leq D$ steps → pseudo-polynomial

RM: three pass, one fails at $41 > 40$

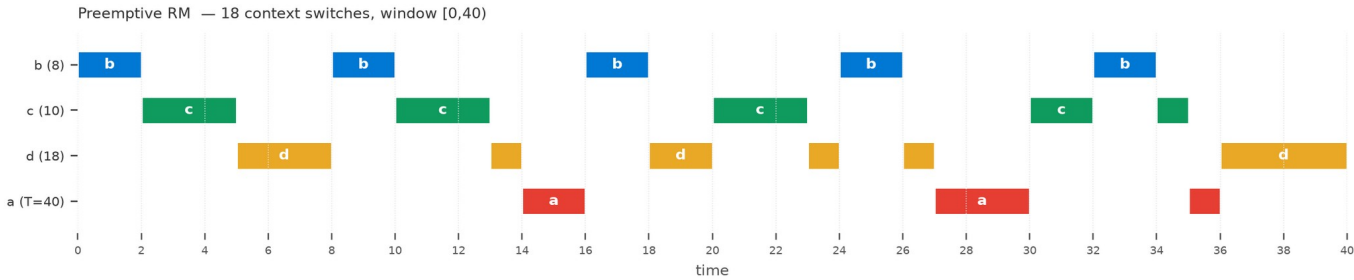
bonus/rta/rta_solver.py • y is baseline (U8g2) but here y is time

RTA ITERATION — FIXED-POINT TRACES

Task	Iter	R	$\leq T?$
τ_b	1: 2	2	$\checkmark 2 \leq 8$
τ_c	2: 5 $\rightarrow$ 5	5	$\checkmark 5 \leq 10$
τ_d	3: 4 $\rightarrow$ 9 $\rightarrow$ 14	14	$\checkmark 14 \leq 18$
τ_a	4: 7 $\rightarrow$ 14 $\rightarrow$ 27 $\rightarrow$ 34 $\rightarrow$ 41	41	$\times 41 > 40$

τ_d example: $R^0=4 \rightarrow R^1=4+\text{ceil}(4/8)\cdot 2+\text{ceil}(4/10)\cdot 3=9 \rightarrow R^2=4+\text{ceil}(9/8)\cdot 2+\text{ceil}(9/10)\cdot 3=14$ stable.
 τ_a : 7 $\rightarrow$ 14 $\rightarrow$ 27 $\rightarrow$ 34 $\rightarrow$ 41 crosses $D=40$ on iteration 4 — RM cannot schedule the full set. σ = interferences from b,c,d stacking.
Exact, not just sufficient. This is why the bloated player isolates Audio (prio 3, core 0).

GANTT — PREEMPTIVE RM OVER [0,40) (bonus/rta/schedule_sim.py $\rightarrow$ RMSim)



C switches	Slots	Idle	Worst slack
18	19	0.7%	τ_d : 4 / 18

Read: a only runs 14–16, 27–30, 35–36 — always preempted. Visual proof of R_i starvation.

SO WHAT FOR THE PLAYER?

RM is optimal among fixed priorities, but this set defeats it. Options:

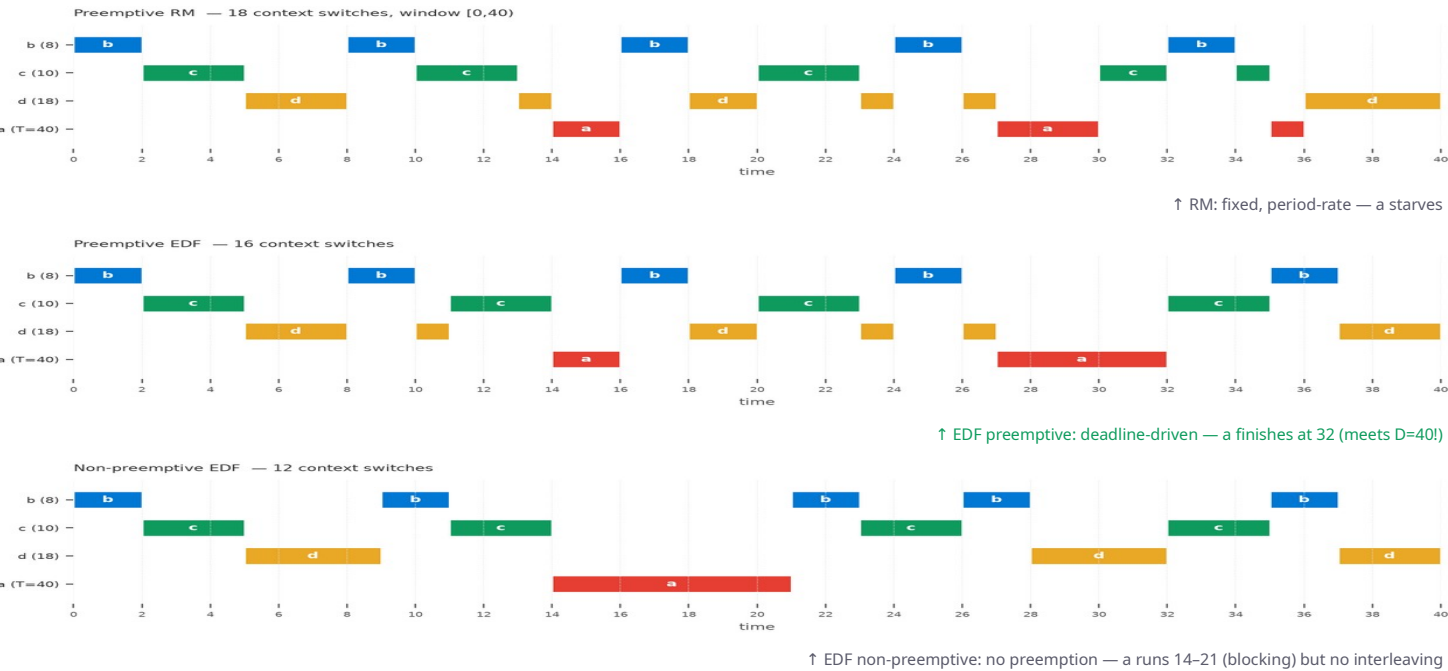
- Drop U (faster CPU, lighter WCET, longer periods) — not possible on ESP32-S3 at 240 MHz with Helix.
- Switch to EDF (dynamic) — next slide.
- Partition cores: we already do (Audio core 0, UI core 1) — formal equivalent of reducing n per core.

*“The Guide says: don’t panic.
The analysis says: τ_a misses
by 1. That’s as real as it gets.”*

RM vs EDF (preemptive) vs EDF (non-preemptive)

Same tasks, same window [0,40) — only the rule changes

GANTTS — VISUAL COMPARISON (all to same scale)



NUMBERS — AT A GLANCE

Scheduler	CS	Meets?	Notes
RM pre.	18	✗ a fails	optimal FP
EDF pre.	16	✓ all meet	dynamic, optimal*
EDF non-pre.	12	✓ but frag.	blocking ↑

EDF optimality*: on uniprocessor with $D \leq T$, if any schedule meets all deadlines, EDF does. Here RM fails, EDF succeeds.

Non-preemptive reduces switches (12 vs 16) but inflates blocking — a must wait for b/c/d to finish (14-21). FreeRTOS is preemptive for a reason.

Player tie: Audio is soft-real-time (glitch = audible). Input/UI are firm. Preemptive EDF intuition justifies Audio prio 3 — it must preempt.

BOTTOM LINE

Fixed priority loses here; dynamic wins. Next: what if tasks share resources? RM/EDF optimality evaporates without a blocking protocol.

R1–R5: PIP vs PCP blocking (bonus/rta/blocking_times.py)

One lock per resource · I²C/SPI/SDMMC are exactly these — priority inversion is not theoretical

CRITICAL-SECTION MATRIX — max CS per task per resource

Task	R1	R2	R3	R4	R5	T
τ _B	—	1	—	1	1	8
τ _C	2	—	1	2	1	10
τ _D	—	4	2	3	—	18
τ _A	6	1	—	—	4	40

Read: τ_A holds R1 for 6 — longest. This drives PCP blocking for everyone else.

WHY THIS MATTERS — INVERSION ON THE WIRE

Without a protocol: low-priority τ_A locks R1 (6 units), medium τ_D/τ_C spin, high τ_B starves — unbounded priority inversion.

- PIP: holder inherits blocked task’s priority → τ_A briefly becomes b-priority (2). But chained blocking sums: B_B = 2+4+4 = 10.
- PCP: ceiling per resource = highest priority that ever uses it. Lock only if priority > all ceilings of locked resources. At most one block: B_B = max(4,3,4)=4.
- Result: PCP halves blocking (10→4, 10→6). τ_D ties because only τ_A can block it — single lower task, sum=max.
- Player mapping: R1/R2... are not abstract — I²C bus (AHT20/BMP280 + 2 LCDs), SPI (ST7565R + font ROM), SDMMC (FATFS). Firmware uses Wire.setTimeout(50 ms) + distinct tasks per bus to bound the same effect.

RESULT — B_i per protocol

Task	B _{PIP}	B _{PCP}	Saving
τ _B	10	4	2.5×
τ _C	10	6	1.7×
τ _D	6	6	no lower task
τ _A	0	0	no lower task

PCP: B = max over resources (max lower CS). PIP: B = sum over lower tasks (max per task). PCP ≤ PIP always.

With PIP/PCP, RTA + B: $R_i = C_i + B_i + \sum \text{ceil}(R_i/T_h) \cdot C_h$ — re-check slides 3–4 adding B_i.
τ_B with B_{PCP}=4 → R=2+4=6 ≤8 still OK; but τ_A’s B=0 — the miss was CPU, not lock. Blocking adds nuance, not the root cause here.

When RM needs a verdict, EDF needs a proof — and that proof can be exponential or clever

Q5/Q6 from the workbook: [bonus/rta/q5_edf_test.py](#) [q6_pdc_complexity.py](#)

Q5 — EDF: PROCESSOR DEMAND CRITERION (Baruah et al. 1990)

τ_1 3/10 τ_2 15/50/200 τ_3 17/100/120 τ_4 200/1000/2000

$$U = \sum C/T$$
$$0.617 \leq 1 \quad \checkmark \text{ necessary}$$

$$h(L) = \sum \text{floor}((L - D_i)/T_i + 1) \cdot C_i$$
$$h(L) \leq L \quad \forall L$$

Need & sufficient: check at every deadline point $L = k \cdot T_i + D_i$ up to H . Solver did 600 points up to $H = \text{lcm}(10, 200, 120, 2000) = 6000$ — all pass: schedulable under EDF.

L	h(L)	L-h(L)	Verdict
10	3	7	✓
100	35	65	✓
1000	235	765	✓
6000	~3680	~2320	✓

... plus 596 more points — all satisfied. See `q5_edf_test.py`

Q6 — THE CATCH: H EXPLODES

Naïve: 600 points to $H=6000$
 H grows exponentially (primes!).

Smart: busy period $L^* = 10$
Only 1 point $\rightarrow 600\times$ fewer.

Iteration: $L^0 = \sum C \rightarrow L^{k+1} = h(L^k)$. Pseudo-polynomial $O(n \cdot D_{\max}/C_{\min})$ — polynomial in values, not bit-length.

WHAT WE'D TELL VOGONS — TAKEAWAYS FOR THE PLAYER

Partition wins

Core 0 = Audio (3) + Input (2) • Core 1 = UI/LED/Matrix/Serial/Vogon (1). Two uniprocessors > one overloaded one. RTA per core is what matters.

Prio = policy

Audio prio 3 is our PCP in practice — I²S cannot wait. Input prio 2 avoids SD-open starvation. Rest 1 = best-effort. Matches RM intuition without being RM.

Bound $\rightarrow$ RTA $\rightarrow$ PDC

Course flow: sufficient ($U \leq \text{bound}$) $\rightarrow$ exact ($\text{RTA} + R_B$) $\rightarrow$ EDF (PDC). Our deck walks that ladder with real numbers.

Complexity is real

$H=6000 \rightarrow 600$ checks; $L^*=10 \rightarrow 1$ check. On ESP32, analysis must be offline — runtime can't afford exponential.

Don't hand-wave stacks

16K Audio/Input came from real canary trips ^{07 / 07}_(_dtoa_r). Schedulability $\neq$ schedulable + stack overflow.

Repro: `bonus/rta/*.py` • rebuild: `bonus/rta/venv/bin/python generate_presentation.py`

So long, and thanks for all the fish.